

Final Model Evaluation

Objective

Compare all three tuned classification models on the locked test set for the first time. This notebook produces the definitive side-by-side comparison that answers the project's central question: which model best predicts railroad accident cause categories using environmental and operational features?

Models Under Evaluation

1. **Logistic Regression** - Linear baseline (L-BFGS, L2, C=1.0, no class weighting)
2. **Random Forest** - Nonlinear benchmark (300 trees, default depth, sqrt features)
3. **CBA (Classification Based on Associations)** - Interpretable rule-based model (undersampled, S=0.005, C=0.40, Lift=1.5)

All three models use the same 5-feature set: Weather Condition, Track Type, Visibility, Region, RUCC_Metro_Adjacency. Accident Type was removed from all models after independent analyses (ARM rule dominance, Cramer's V, RF Gini importance, L1 sparsity) confirmed it acts as a dominant confounding variable.

Test Set Methodology

The test set (20% of data, ~6,911 rows) was created during Phase 2 using stratified sampling with random_state=521. It has been locked since creation - no model selection, feature selection, or hyperparameter decision has been influenced by this data. This is the first and only time it will be opened.

Metrics

- **Primary:** Weighted F1-score
 - **Secondary:** Per-class Precision, Recall, F1
 - **Tertiary:** Confusion matrices
 - **Additional:** PR curves (LR and RF only), cross-model error analysis
-

Setup

```
In [ ]: import pandas as pd
import numpy as np
```

```

import json
import joblib
import warnings

from sklearn.metrics import (
    classification_report, f1_score, confusion_matrix,
    precision_score, recall_score, precision_recall_curve,
    average_precision_score
)

import matplotlib.pyplot as plt
import matplotlib.style as style
import seaborn as sns

warnings.filterwarnings('ignore', category=FutureWarning)
warnings.filterwarnings('ignore', category=UserWarning)
style.use('tableau-colorblind10')

plt.rcParams['figure.dpi'] = 120
plt.rcParams['font.size'] = 10

RANDOM_SEED = 521
np.random.seed(RANDOM_SEED)

CAUSE_ORDER = ['E', 'H', 'M', 'S', 'T']
CAUSE_LABELS = {
    'E': 'Environmental', 'H': 'Human Factor',
    'M': 'Mechanical', 'S': 'Signal', 'T': 'Track'
}

print("Setup complete.")

```

Load Test Set

This cell loads the locked test set. It has not been used for any model selection, feature selection, or hyperparameter tuning decisions. Uncomment and run only when ready for final evaluation.

```

In [ ]: # =====
# UNCOMMENT WHEN READY TO RUN FINAL EVALUATION
# =====
# df_test = pd.read_csv('../data/splits/test_20.csv', low_memory=False)
#
# target = 'Cause_Category'
# y_test = df_test[target].copy()
#
# # Same 5 features used by all three models
# feature_cols = ['Weather Condition', 'Track Type', 'Visibility', 'Region',
#                 'RUCC_Metro_Adjacency']
# X_test = df_test[feature_cols].copy()
#
# print(f"Test set: {len(df_test)} rows")
# print(f"\nCause_Category distribution:")

```

```
# print(y_test.value_counts())
# print(f"\nDistribution (%):")
# print(y_test.value_counts(normalize=True).round(3))
```

Load Tuned Models

```
In [ ]: # =====
# UNCOMMENT WHEN READY TO RUN FINAL EVALUATION
# =====
# lr_pipeline = joblib.load('../models/lr_tuned_pipeline.pkl')
# rf_pipeline = joblib.load('../models/rf_tuned_pipeline.pkl')
# cba_model = joblib.load('../models/cba_tuned_model.pkl')
#
# print("LR pipeline loaded:", type(lr_pipeline))
# print("RF pipeline loaded:", type(rf_pipeline))
# print("CBA model loaded:", type(cba_model))
# print(f"CBA rules: {cba_model.n_rules}")
```

```
In [ ]: # Load tuning experiment results (CV scores from training)
# These are available now for framework building

def load_json(filepath):
    with open(filepath, 'r') as f:
        return json.load(f)

lr_cv = load_json('../results/lr/lr_tuned_gridsearch.json')
rf_cv = load_json('../results/rf/rf_tuned_gridsearch.json')
cba_cv = load_json('../results/cba/cba_tuned_undersampled.json')

print("CV Results (from training):")
print(f" LR:  {lr_cv['cv_mean']:.4f} +/- {lr_cv['cv_std']:.4f}")
print(f" RF:  {rf_cv['cv_mean']:.4f} +/- {rf_cv['cv_std']:.4f}")
print(f" CBA: {cba_cv['cv_mean']:.4f} +/- {cba_cv['cv_std']:.4f}")
```

Section 1: Test Set Evaluation

Run each tuned model on the test set and collect predictions. Each model sees the exact same test data, producing directly comparable results.

```
In [ ]: # =====
# UNCOMMENT WHEN READY TO RUN FINAL EVALUATION
# =====
# # Generate predictions
# y_pred_lr = lr_pipeline.predict(X_test)
# y_pred_rf = rf_pipeline.predict(X_test)
# y_pred_cba = cba_model.predict(X_test)
#
# # Calculate weighted F1 for each
# f1_lr = f1_score(y_test, y_pred_lr, average='weighted')
```

```
# f1_rf = f1_score(y_test, y_pred_rf, average='weighted')
# f1_cba = f1_score(y_test, y_pred_cba, average='weighted')
#
# print("Test Set Weighted F1:")
# print(f" LR: {f1_lr:.4f}")
# print(f" RF: {f1_rf:.4f}")
# print(f" CBA: {f1_cba:.4f}")
```

Per-Class Metrics

Weighted F1 summarizes overall performance, but per-class metrics reveal how each model handles the class imbalance problem. Signal (2.5%) and Environmental (11%) are the critical minority classes.

```
In [ ]: # =====
# UNCOMMENT WHEN READY TO RUN FINAL EVALUATION
# =====
# def get_per_class_metrics(y_true, y_pred, model_name):
#     """Build per-class metrics DataFrame for one model."""
#     p = precision_score(y_true, y_pred, average=None, labels=CAUSE_ORDER, zero_divide=0)
#     r = recall_score(y_true, y_pred, average=None, labels=CAUSE_ORDER, zero_divide=0)
#     f = f1_score(y_true, y_pred, average=None, labels=CAUSE_ORDER, zero_divide=0)
#
#     return pd.DataFrame({
#         'Cause': CAUSE_ORDER,
#         'Precision': p.round(3),
#         'Recall': r.round(3),
#         'F1': f.round(3),
#         'Model': model_name
#     })
#
# metrics_lr = get_per_class_metrics(y_test, y_pred_lr, 'LR')
# metrics_rf = get_per_class_metrics(y_test, y_pred_rf, 'RF')
# metrics_cba = get_per_class_metrics(y_test, y_pred_cba, 'CBA')
#
# print("LR Per-Class:")
# print(metrics_lr[['Cause', 'Precision', 'Recall', 'F1']].to_string(index=False))
# print(f"\nRF Per-Class:")
# print(metrics_rf[['Cause', 'Precision', 'Recall', 'F1']].to_string(index=False))
# print(f"\nCBA Per-Class:")
# print(metrics_cba[['Cause', 'Precision', 'Recall', 'F1']].to_string(index=False))
```

```
In [ ]: # =====
# UNCOMMENT WHEN READY TO RUN FINAL EVALUATION
# =====
# def save_test_results(model_name, f1_test, per_class_df, cv_data, filepath):
#     output = {
#         'experiment': f'{model_name} Final Test Evaluation',
#         'model': model_name,
#         'test_f1': float(f1_test),
#         'cv_mean': cv_data['cv_mean'],
#         'cv_std': cv_data['cv_std'],
#         'cv_test_delta': float(f1_test - cv_data['cv_mean']),
#         'test_per_class_metrics': per_class_df[['Cause', 'Precision', 'Recall', 'F1']]
```

```
#         'random_seed': RANDOM_SEED
#     }
#     with open(filepath, 'w') as f:
#         json.dump(output, f, indent=4)
#     print(f"Saved: {filepath}")
#
# save_test_results('Logistic Regression', fl_lr, metrics_lr, lr_cv,
#                  './results/lr/lr_test_results.json')
# save_test_results('Random Forest', fl_rf, metrics_rf, rf_cv,
#                  './results/rf/rf_test_results.json')
# save_test_results('CBA', fl_cba, metrics_cba, cba_cv,
#                  './results/cba/cba_test_results.json')
```

Section 2: Cross-Model Comparison

Summary Table

The most important table in the project. CV Mean shows training performance, Test F1 shows generalization, and the delta between them indicates whether models overfit or underfit during training.

```
In [ ]: # =====
# UNCOMMENT WHEN READY TO RUN FINAL EVALUATION
# =====
# summary = pd.DataFrame({
#     'Model': ['Logistic Regression', 'Random Forest', 'CBA (Undersampled)'],
#     'CV Mean': [lr_cv['cv_mean'], rf_cv['cv_mean'], cba_cv['cv_mean']],
#     'CV Std': [lr_cv['cv_std'], rf_cv['cv_std'], cba_cv['cv_std']],
#     'Test F1': [fl_lr, fl_rf, fl_cba],
#     'Delta (Test - CV)': [fl_lr - lr_cv['cv_mean'],
#                          fl_rf - rf_cv['cv_mean'],
#                          fl_cba - cba_cv['cv_mean']]
# })
#
# print("=" * 75)
# print(f"{'Model':<25} {'CV Mean':>10} {'CV Std':>10} {'Test F1':>10} {'Delta (Test - CV)':>10}")
# print("-" * 75)
# for _, row in summary.iterrows():
#     print(f"{'Model':<25} {'CV Mean':>10.4f} {'CV Std':>10.4f} {'Test F1':>10.4f} {'Delta (Test - CV)':>10.4f}")
# print("=" * 75)
```

Per-Class F1 Comparison

This chart answers: which model handles which cause category best? Pay attention to Signal and Environmental - these are the classes that CBA's undersampling was designed to improve.

```
In [ ]: # =====
# UNCOMMENT WHEN READY TO RUN FINAL EVALUATION
# =====
# fig, ax = plt.subplots(figsize=(8, 5))
#
# x = np.arange(len(CAUSE_ORDER))
# width = 0.25
# hatches = ['//', '..', 'xx']
#
# models = [('LR', metrics_lr), ('RF', metrics_rf), ('CBA', metrics_cba)]
#
# for i, (name, df) in enumerate(models):
#     offset = (i - 1) * width
#     bars = ax.bar(x + offset, df['F1'], width, label=name)
#     for bar in bars:
#         bar.set_hatch(hatches[i])
#
# ax.set_xlabel('Cause Category')
# ax.set_ylabel('F1-Score')
# ax.set_title('Per-Class F1: All Models on Test Set')
# ax.set_xticks(x)
# ax.set_xticklabels([f"{c} ({CAUSE_LABELS[c]})" for c in CAUSE_ORDER], rotation=45)
# ax.legend()
# ax.set_ylim(0, 1.0)
# plt.tight_layout()
# plt.show()
```

Confusion Matrices

Side-by-side confusion matrices reveal each model's error patterns. Look for:

- **Diagonal strength:** Which model has the most predictions on the diagonal (correct)?
- **Off-diagonal patterns:** Where does each model misclassify? Do they confuse the same pairs?
- **Column sparsity:** Does the model even attempt to predict certain classes?

```
In [ ]: # =====
# UNCOMMENT WHEN READY TO RUN FINAL EVALUATION
# =====
# fig, axes = plt.subplots(1, 3, figsize=(8, 5))
#
# predictions = [
#     ('LR', y_pred_lr),
#     ('RF', y_pred_rf),
#     ('CBA', y_pred_cba)
# ]
#
# for ax, (name, y_pred) in zip(axes, predictions):
#     cm = confusion_matrix(y_test, y_pred, labels=CAUSE_ORDER)
#     sns.heatmap(cm, annot=True, fmt='d', cmap='cividis',
#                 xticklabels=CAUSE_ORDER, yticklabels=CAUSE_ORDER, ax=ax)
```

```

#     ax.set_title(name)
#     ax.set_ylabel('True' if name == 'LR' else '')
#     ax.set_xlabel('Predicted')
#
# fig.suptitle('Confusion Matrices: Test Set', fontsize=12, y=1.02)
# plt.tight_layout()
# plt.show()

```

Precision-Recall Curves (LR and RF)

PR curves show the precision-recall tradeoff across all decision thresholds. They are more informative than ROC curves for imbalanced datasets because they focus on the positive class performance without being inflated by true negatives. One PR curve per cause category, per model.

CBA is excluded because it uses first-match rule classification and does not produce probability estimates.

```

In [ ]: # =====
# UNCOMMENT WHEN READY TO RUN FINAL EVALUATION
# =====
# from sklearn.preprocessing import label_binarize
#
# # Binarize test labels for PR curves
# y_test_bin = label_binarize(y_test, classes=CAUSE_ORDER)
#
# # Get probability estimates
# y_prob_lr = lr_pipeline.predict_proba(X_test)
# y_prob_rf = rf_pipeline.predict_proba(X_test)
#
# fig, axes = plt.subplots(1, 5, figsize=(8, 4))
#
# for i, cause in enumerate(CAUSE_ORDER):
#     ax = axes[i]
#
#     # LR
#     p_lr, r_lr, _ = precision_recall_curve(y_test_bin[:, i], y_prob_lr[:, i])
#     ap_lr = average_precision_score(y_test_bin[:, i], y_prob_lr[:, i])
#     ax.plot(r_lr, p_lr, linewidth=1.5, label=f'LR (AP={ap_lr:.2f})')
#
#     # RF
#     p_rf, r_rf, _ = precision_recall_curve(y_test_bin[:, i], y_prob_rf[:, i])
#     ap_rf = average_precision_score(y_test_bin[:, i], y_prob_rf[:, i])
#     ax.plot(r_rf, p_rf, linewidth=1.5, linestyle='--', label=f'RF (AP={ap_
#
#     ax.set_title(f'{cause}')
#     ax.set_xlim(0, 1)
#     ax.set_ylim(0, 1)
#     ax.legend(fontsize=6)
#     if i == 0:
#         ax.set_ylabel('Precision')
#     ax.set_xlabel('Recall')
#
#

```

```
# fig.suptitle('Precision-Recall Curves by Cause Category', fontsize=12, y=1
# plt.tight_layout()
# plt.show()
```

Section 3: Error Analysis

Understanding where models fail is as valuable as knowing where they succeed. This section examines prediction agreement, shared errors, and feature patterns in misclassified rows.

Model Agreement

How often do all three models agree on a prediction? When they disagree, which models tend to align?

```
In [ ]: # =====
# UNCOMMENT WHEN READY TO RUN FINAL EVALUATION
# =====
# # Build agreement DataFrame
# agreement = pd.DataFrame({
#     'true': y_test.values,
#     'lr': y_pred_lr,
#     'rf': y_pred_rf,
#     'cba': y_pred_cba
# })
#
# # All three agree
# all_agree = (agreement['lr'] == agreement['rf']) & (agreement['rf'] == agr
# all_correct = all_agree & (agreement['lr'] == agreement['true'])
# all_wrong = all_agree & (agreement['lr'] != agreement['true'])
#
# print(f"All three models agree: {all_agree.sum()} / {len(agreement)} ({all
# print(f" All agree AND correct: {all_correct.sum()} ({all_correct.mean():
# print(f" All agree AND wrong: {all_wrong.sum()} ({all_wrong.mean():.1%}
# print(f" At least one disagrees: {(~all_agree).sum()} ({(~all_agree).mean
#
# # Pairwise agreement
# lr_rf = (agreement['lr'] == agreement['rf']).mean()
# lr_cba = (agreement['lr'] == agreement['cba']).mean()
# rf_cba = (agreement['rf'] == agreement['cba']).mean()
#
# print(f"\nPairwise agreement:")
# print(f" LR-RF: {lr_rf:.1%}")
# print(f" LR-CBA: {lr_cba:.1%}")
# print(f" RF-CBA: {rf_cba:.1%}")
```

Shared Errors

When all three models make the same wrong prediction, the error is likely caused by genuinely ambiguous data rather than a model weakness. Examining these cases reveals the limits of what 5 categorical features can distinguish.

```
In [ ]: # =====  
# UNCOMMENT WHEN READY TO RUN FINAL EVALUATION  
# =====  
# # Rows where all three agree but are wrong  
# shared_errors = agreement[all_wrong].copy()  
# shared_errors_full = df_test.loc[shared_errors.index, feature_cols + [target]]  
# shared_errors_full['predicted'] = shared_errors['lr'].values  
#  
# print(f"Shared errors: {len(shared_errors_full)} rows")  
# print(f"\nTrue vs Predicted distribution in shared errors:")  
# error_crosstab = pd.crosstab(shared_errors_full[target], shared_errors_full[predicted],  
#                               margins=True)  
# print(error_crosstab)  
#  
# print(f"\nMost common feature combinations in shared errors:")  
# top_patterns = shared_errors_full.groupby(feature_cols).size().sort_values(ascending=False)  
# print(top_patterns)
```

Section 4: The Accident Type Story

One of the central findings of this project is the identification and removal of Accident Type as a dominant confounding variable. This section summarizes the evidence and quantifies the impact across all three models.

Evidence for Removal

Three independent analyses converged on the same conclusion:

1. **ARM Rule Mining (Phase 2):** When Accident Type was included, all rules reduced to Accident Type to Cause mappings (e.g., Hwy-rail crossing to Mechanical). Without it, ARM found zero rules at standard thresholds, proving Accident Type was masking all other feature relationships.
2. **Cramer's V (Phase 2):** Accident Type showed 0.39 association with Cause_Category, nearly double the next strongest feature (Track Type at 0.21). This level of association in a categorical feature is a red flag for confounding.
3. **L1 Sparsity (LR Tuning):** With Accident Type included, 7 of the 10 largest model coefficients belonged to Accident Type categories. Hwy-rail crossing (0.60) was 3x the magnitude of the strongest non-Accident-Type feature.

4. **RF Gini Importance (RF Baseline):** Accident Type consumed ~0.48 of the total importance budget, leaving all other features compressed below 0.15.

Impact Across Models

```
In [ ]: # Load baseline results for comparison
lr_baseline = load_json('../results/lr/lr_baseline_results.json')
rf_baseline = load_json('../results/rf/rf_baseline_results.json')
cba_baseline = load_json('../results/cba/cba_baseline_results.json')

print("Impact of Removing Accident Type (CV scores, training data)")
print("=" * 70)
print(f"{'Model':<10} {'With AT':>12} {'Without AT':>12} {'Drop':>10} {'Drop'")
print("-" * 70)

models = [
    ('LR', lr_baseline['cv_mean'], lr_cv['cv_mean']),
    ('RF', rf_baseline['cv_mean'], rf_cv['cv_mean']),
    ('CBA', cba_baseline['cv_mean'], cba_cv['cv_mean'])
]

for name, with_at, without_at in models:
    drop = without_at - with_at
    drop_pct = drop / with_at * 100
    print(f"{'name':<10} {'with_at':>12.4f} {'without_at':>12.4f} {'drop':>+10.4f} {"

print("=" * 70)
print("\nAll three models lost 0.08-0.13 F1 when Accident Type was removed.")
print("This consistency across linear, tree-based, and rule-based algorithms")
print("confirms Accident Type dominance is a data-level phenomenon, not mode
```

Section 5: CBA Rule Interpretability

CBA's unique value proposition is that it produces human-readable classification rules. While LR and RF may slightly outperform CBA on weighted F1, neither can explain individual predictions as explicit if-then rules. For railroad safety applications, this interpretability has direct operational value.

The rules below were mined from undersampled training data using Apriori with support=0.005, confidence=0.40, and lift=1.5. Each rule describes a combination of environmental and operational conditions associated with a specific cause category.

```
In [ ]: # =====
# UNCOMMENT WHEN READY TO RUN FINAL EVALUATION
# =====
# print(f"Total rules: {cba_model.n_rules}")
# print(f"Default class: {cba_model.default_class} ({CAUSE_LABELS[cba_model
```

```
# print(f"\nRules by predicted cause:")
# print(cba_model.rules_['predicted_cause'].value_counts())
#
# print(f"\nTop 20 rules:")
# top_rules = cba_model.rules_[['antecedents', 'predicted_cause', 'confidence',
#                               'support', 'lift']].head(20).copy()
# top_rules['antecedents'] = top_rules['antecedents'].apply(lambda x: ', '.join(x))
# print(top_rules.to_string(index=False))
```

Interpreting the Rules

Each rule has three quality metrics:

- **Confidence:** The percentage of times this rule correctly predicts the cause. A rule with 0.65 confidence is correct 65% of the time when its conditions are met.
- **Support:** How frequently this pattern appears in the (undersampled) training data. Higher support means the pattern is more common and the rule is based on more evidence.
- **Lift:** How much better this rule performs compared to random guessing. A lift of 3.0 means the rule is 3x better than predicting the cause by chance.

Example interpretation: "Track Type_Industry, Weather Condition_Cloudy, Visibility_Day -> Track (confidence 0.70, lift 3.5)" means that when an accident occurs on industry track, in cloudy weather, during daytime, there is a 70% chance it was caused by a Track failure, and this association is 3.5x stronger than random.

Section 6: Conclusions and Recommendations

Model Comparison Summary

[Fill in after test results]

Key Findings

[Fill in after test results]

1. **Overall performance:** *[Which model won on test F1? By how much?]*
2. **Generalization:** *[Did CV scores align with test scores? Any overfitting?]*
3. **Minority class handling:** *[Which model best detected Signal and Environmental?]*
4. **Interpretability:** *[CBA's unique contribution]*
5. **The Accident Type finding:** *[Central methodological contribution]*

Which Model Would We Deploy?

[Fill in - this may depend on the use case:]

- *If maximizing overall accuracy: [model]*
- *If detecting rare Signal/Environmental causes matters: [model]*
- *If the end user needs to understand why a prediction was made: [model]*

Limitations

[Fill in]

- 5 categorical features with limited discriminative power for rare classes
- Class imbalance (Signal at 2.5%) remains a fundamental challenge
- Accident Type removal was methodologically correct but reduced all models to ~0.41-0.42 F1
- CBA's undersampling introduces variance through random balanced subsets

Future Work

[Fill in]

- Entity embeddings (Guo & Berkhahn, 2016) to learn dense representations of categorical features
- Additional features from the FRA dataset (speed, tonnage, time of day)
- Integration with Matt and Tim's regression models for damage cost prediction

Integration with Regression Analysis

[Fill in after coordinating with teammates - how do classification and regression findings complement each other?]